

II.1 Files

As mentioned in the previous volume, on a Linux operating system everything is a file. A file is a structure that contains data and information about that data. In the C Library there are functions that allow you to read the information of the files and others that allow you to modify the properties of the files. In Linux there are different types of files, e.g. to manage a device (terminal, keyboard, screen, etc..) and to represent it in the file system it is used a different type of file than the one used to represent a simple text (or data) file. Each file in a Linux file system belongs to a particular user and group, has a creation date and a last modification date and has a set of permissions that define who and how can access the file and what operation can be performed on it (read, write, execute). A directory is a point of access in the file system and within it there can be files or directories. However a directory also is a file like everything else in a Linux fs.

II.1.1 Stat of files: `stat`, `fstat` and `lstat`

To retrieve the information about a file, the following functions are used.

```
#include <sys/stat.h>

int stat(const char *restrict pathname, struct stat *restrict statbuf);

int fstat(int fd, struct stat *statbuf);

int lstat(const char *restrict pathname, struct stat *restrict
          statbuf);

#include <fcntl.h>
#include <sys/stat.h>

int fstatat(int dirfd, const char *restrict pathname, struct stat
            *restrict statbuf, int flags);

Return: on success 0, on error -1 and errno
```

All these functions return information about a file through a buffer called `statbuf`. To access information about a file, a program must have permission to traverse every directory in the file's path. The required permission is execute permission, since traversing a directory requires the user or program to *execute* that directory.¹ `stat`, `fstatat` and `lstat` need these permissions to reach the file and read the metadata.

`stat` and `fstatat` retrieve the information about the file pointed to by `pathname`.

`lstat` is identical to `stat`, except that if `pathname` refers to a symbolic link, it returns information about the link itself rather than about the file to which the link points.

`fstat` is identical to `stat` except that the file is specified by a file descriptor (`fd`).

All these functions return information in a `stat`² structure (`statbuf`).

1 To execute a directory means literally 'go inside the directory', to read a directory means 'read the content of the directory'

2 See the `stat(3)` manual page

Since the purpose of these functions is to retrieve information about a file, the structure used to store the information and the pathname itself must each be accessed through a single pointer. For this reason, `pathname` and `statbuf` are declared with the `restrict`³ qualifier. The `dirfd` argument of `fstatat` determines how the file path is resolved according to the following rules:

- if `pathname` is relative, it is interpreted relative to the directory specified in `dirfd` rather than relative to the current working directory
- if `pathname` is relative and `dirfd` has the special value `AT_FDCWD` then `pathname` is interpreted relative to the current working directory
- If `pathname` is an absolute pathname `dirfd` is ignored

The `flags` argument can be 0 or one of the following values:

<code>AT_EMPTY_PATH</code>	If <code>pathname</code> is an empty string the function will operate on the file referred to by <code>dirfd</code> . This last in this case can refer to any type of file. If <code>dirfd</code> is <code>AT_FDCWD</code> the function operate on the current working directory.
<code>AT_NO_AUTOMOUNT</code>	Don't automount the terminal ("basename") component of <code>pathname</code>
<code>AT_SYMLINK_NOFOLLOW</code>	If <code>pathname</code> is a symbolic link do not dereference it and return information on the link itself

Tab 1. Values of the `flags` argument of `fstatat`

The possible errors returned in `errno` are the following:

<code>EACCESS</code>	search permission is denied for one of the directories in the path
<code>EBADF</code>	<code>fd</code> is not a valid file descriptor
<code>EBADF</code> (<code>fstatat</code>)	<code>pathname</code> is relative but <code>dirfd</code> is not <code>AT_FDCWD</code> nor a valid descriptor
<code>EFAULT</code>	bad address
<code>EINVAL</code> (<code>fstatat</code>)	<code>flags</code> has not a valid value
<code>ELOOP</code>	too many symbolic links encountered while traversing the path
<code>ENAMETOOLONG</code>	<code>pathname</code> is too long
<code>ENOENT</code>	a component of <code>pathname</code> does not exist or is a not valid symbolic link
<code>ENOENT</code> (<code>fstatat</code>)	<code>pathname</code> is an empty string and <code>AT_EMPTY_PATH</code> is not specified
<code>ENOMEM</code>	out of memeory
<code>ENOTDIR</code>	a component of <code>pathname</code> is not a directory
<code>ENOTDIR</code> (<code>fstatat</code>)	<code>pathname</code> is relative and <code>dirfd</code> is a file descriptor referring to a file other than a directtory
<code>EOVERFLOW</code>	<code>pathname</code> or <code>fd</code> refers to a file whose size, inode number, or number of blocks cannot be represented in, respectively, the types <code>off_t</code> ,

3 About the use of the keyword `restrict` consult the *C/99 Standard*, the link is in the bibliography

	ino_t, blkcnt_t ⁴
--	------------------------------

Tab 2. errno values for the stat functions

The information retrieved from the file is stored in a structure of type `stat`⁵. This structure represents the status of the file.

```
#include <sys/stat.h>

struct stat {
    dev_t st_dev;           /* ID of device containing file */
    ino_t st_ino;           /* Inode number */
    mode_t st_mode;         /* File type and mode */
    nlink_t st_nlink;       /* Number of hard links */
    uid_t st_uid;           /* User ID of owner */
    gid_t st_gid;           /* Group ID of owner */
    dev_t st_rdev;          /* Device ID (if special file) */
    off_t st_size;          /* Total size, in bytes */
    blksize_t st_blksize;   /* Block size for filesystem I/O */
    blkcnt_t st_blocks;     /* Number of 512B blocks allocated */

    /* Since POSIX.1-2008, this structure supports nanosecond
       precision for the following timestamp fields */

    struct timespec st_atim; /* Time of last access */
    struct timespec st_mtim; /* Time of last modification */
    struct timespec st_ctim; /* Time of last status change */

    #define st_atime st_atim.tv_sec /* Backward compatibility */
    #define st_mtime st_mtim.tv_sec
    #define st_ctime st_ctim.tv_sec
};
```

The `stat` structure is composed by the following fields:

<code>st_dev</code>	device on which the file resides
<code>st_ino</code>	the inode ⁶ number of the file
<code>st_mode</code>	type and mode of the file
<code>st_nlink</code>	number of hard links to the file
<code>st_uid</code>	user ID of owner of the file
<code>st_gid</code>	group ID of the owner of the file
<code>st_rdev</code>	device represented by the file

4 See the `stat(2)` manual page for an example of this error and the manual pages: `off_t(3type)`, `stat(3type)`, `blkcnt_t(3type)` for the types

5 See the `stat(3type)` manual page

6 An inode is a number that identifies a file on the file system, each file and directory has his unique inode number

st_size	size of the file
st_blksize	block size
st_blocks	number of blocks allocated to the file (1 block = 512 bytes)
st_atime	time of last access to the file data
st_mtime	time of last modification of file data
st_ctime	file's last status change timestamp

Tab 3. Fields of the `stat` structure

The other fields (`st_atim.tv_sec`, `st_mtim.tv_sec`, and `st_ctim.tv_sec`) are provided for backward compatibility. The `st_ino` field represents the inode number of the file. An *inode* is a number that uniquely identifies a file within the filesystem. Every file and directory in a Linux filesystem has its own *inode*. On Linux systems, information about a file can be displayed using the `stat` program. When a symbolic link is created, the link stores a reference to the pathname of another file rather than sharing the same *inode*.

We create a file and write some data.

```
[linux@fidir] touch file.txt
[linux@fidir] echo 'hello' > file.txt
```

Then we create a symbolic link to the file with `ln` and the option `-s`

```
[linux@fidir] ln -s file.txt file2
[linux@fidir] ls -l
total 4
lrwxrwxrwx 1 fidir fidir 8 Mar 23 19:58 file2 -> file.txt
-rw-r--r-- 1 fidir fidir 6 Mar 23 19:58 file.txt
```

As we can see, the link `file2` point to the file `file.txt`. If we run `cat` on `file2` we see the content of `file.txt`

```
[linux@fidir] cat file2
hello
```

To retrieve information about the file we use the `stat` command.

```
[linux@fidir] stat file.txt
File: file.txt
Size: 6      Blocks: 8      IO Block: 4096  regular file
Device: 259,2 Inode: 23364606 Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/  fidir)  Gid: ( 1000/  fidir)
Access: 2024-03-23 20:01:37.452040888 +0700
Modify: 2024-03-23 19:58:18.396035269 +0700
Change: 2024-03-23 19:59:46.536037757 +0700
Birth: 2024-03-23 19:58:11.664035079 +0700
```

The output of `stat` shows detailed file metadata, including file size, number of blocks used, file type, device ID, inode number, number of hard links, permissions, ownership (UID/GID), and timestamps for last access, modification, change, and creation (birth time if supported).

If we break down:

File	name of the file
Size	size in bytes
Blocks	number of filesystem blocks allocated
IO Block	preferred block size for I/O operations
Device	device identifier where the file resides
Inode	unique identifier of the file in the filesystem
Links	number of hard links pointing to this inode
Access	file type and permission bits
Uid/Gid	owner user and group
Timestamps:	
Access	last time file was read
Modify	last time content was changed
Change	last time metadata (permissions, ownership, etc.) changed
Birth	creation time (not always available on all filesystems)

So overall, `stat` is essentially a “full identity card” for a file in Linux.

We run `stat` on the symbolic link `file2`

```
[linux@fidir] stat file2
File: file2 -> file.txt
Size: 8      Blocks: 0      IO Block: 4096  symbolic link
Device: 259,2 Inode: 23364884  Links: 1
Access: (0777/lrwxrwxrwx) Uid: ( 1000/  fidir)  Gid: ( 1000/  fidir)
Access: 2024-03-23 19:59:54.840037991 +0700
Modify: 2024-03-23 19:58:38.512035836 +0700
Change: 2024-03-23 19:59:46.536037757 +0700
Birth: 2024-03-23 19:58:38.512035836 +0700
```

The inode numbers are different because a symbolic link is treated as a separate file in the filesystem, with its own inode. A symbolic link acts as a pointer to another file, but it is still considered an independent filesystem object.

If inode 23364884 is removed, the symbolic link `file2` is deleted, while `file.txt` remains unchanged. Conversely, if inode 23364606 (the target file) is deleted, `file2` is not removed, but it becomes a broken (*dangling*) *symbolic link* that points to a non-existent file.

The `Links` field in the output does not refer to symbolic links; it refers to *hard links*. A *hard link* is a reference to a file that, unlike a *symbolic link*, is not a separate filesystem entity. Multiple hard links can point to the same *inode*, meaning a single inode number may correspond to multiple filenames.

For this reason, a symbolic link has its own *inode number*, different from the target file, while a *hard link* shares the same inode number as the original file.

We create a hard link to the file `file.txt` (command `ln` without options) and execute `stat`

```
[linux@fidir] ln file.txt hfile3
[linux@fidir] stat hfile3
File: hfile3
Size: 6      Blocks: 8      IO Block: 4096   regular file
Device: 259,2 Inode: 23364606  Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/   fidir)   Gid: ( 1000/   fidir)
Access: 2024-03-23 20:01:37.452040888 +0700
Modify: 2024-03-23 19:58:18.396035269 +0700
Change: 2024-03-23 20:08:06.860051880 +0700
Birth:   2024-03-23 19:58:11.664035079 +0700
```

The inode number of `hfile3` is the same as that of `file.txt`, and number of links is 2 because the same inode is associated with two filenames: `file.txt` and `hfile3`. However, if we execute `ls -l`, we can see that `hfile3` appears as a separate file and does not point to `file.txt`.

```
[linux@fidir] ls -l
total 8
lrwxrwxrwx 1 fidir fidir 8 Mar 23 19:58 file2 -> file.txt
-rw-r--r-- 2 fidir fidir 6 Mar 23 19:58 file.txt
-rw-r--r-- 2 fidir fidir 6 Mar 23 19:58 hfile3
```

In reality, `hfile3` is not a different file, but another reference to the same file. Even though it appears as a separate file in the output of `ls`, it is simply another filename used to access the same underlying data and inode.

We add some text to `file.txt` and look at the content of `hfile3`

```
[linux@fidir] echo 'world' >> file.txt
[linux@fidir] cat hfile3
hello
world
```

When we write to `file.txt` we are effectively telling Linux: "Write this text into the file identified by inode number 23364606". Since this inode has two references (two filenames that refer to the same data), Linux updates the data associated with that inode, and both references will then access the updated content.

If we execute `cat` on the symbolik link `file2`, it will display the contents of `file.txt`. However, `file2` is not a direct reference to `file.txt` in the same way a hard link is. Instead, `file2` is a separate file whose inode contains a pointer to the path of another file. In other words, `file2` is itself a filesystem object (with its own inode), and that inode stores information that directs the operating system to `file.txt`. Therefore, unlike a hard link, a symbolic link does not share the same inode or data as the target file.

The following picture shows this relationship.

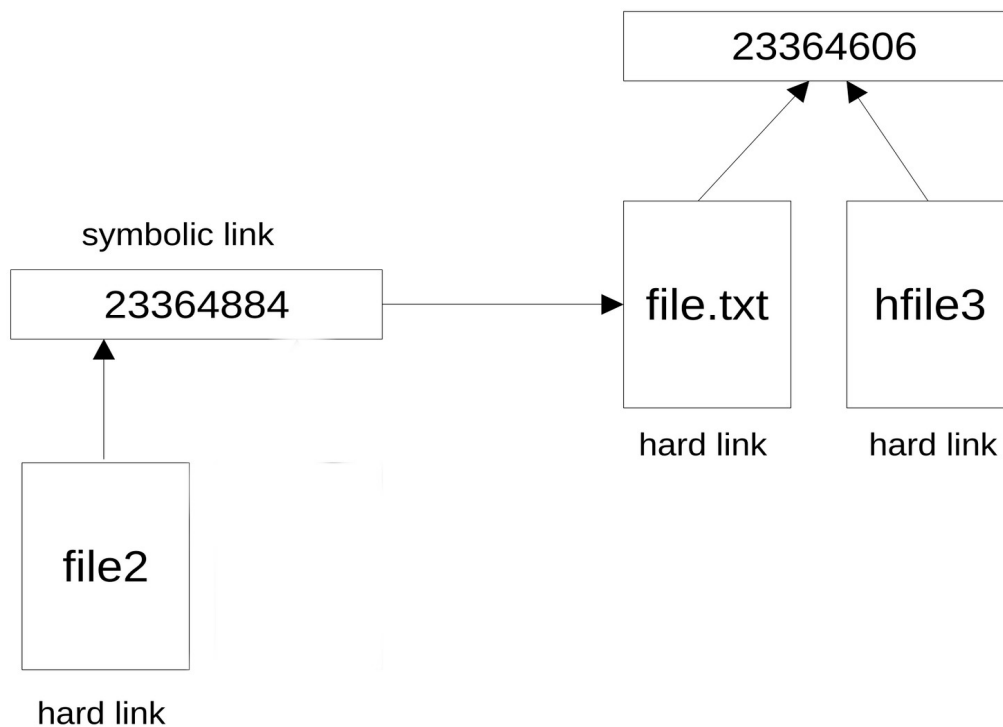


Fig 1. Relationship between hard links and symbolic links

`file2` is an hard link to the inode `23364884` that is a symbolic link to `file.txt` that is an hard link to the inode `23364606`. `hfile3` is another hard link to the same inode. We can also create another hard link to the inode `23364884`.

```
[linux@fidir] ln file2 sfile4
[linux@fidir] ls -l
total 8
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 file2 -> file.txt
-rw-r--r-- 2 fidir fidir 6 Mar 23 20:38 file.txt
-rw-r--r-- 2 fidir fidir 6 Mar 23 20:38 hfile3
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 sfile4 -> file.txt
```

`sfile4` is another symbolic link to `file.txt`, this because we create an hard link that is a reference to an inode that points to another file (`file.txt`).

We run `stat` on `sfile4`

```
[linux@fidir] stat sfile4
File: sfile4 -> file.txt
Size: 8      Blocks: 0      IO Block: 4096  symbolic link
Device: 259,2 Inode: 23364884  Links: 2
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/  fidir)  Gid: ( 1000/  fidir)
Access: 2024-03-23 20:53:24.412128593 +0700
Modify: 2024-03-23 20:38:10.584102797 +0700
Change: 2024-03-23 20:53:22.808128548 +0700
Birth: 2024-03-23 20:38:10.584102797 +0700
```

The inode number is the same of `file2`. The following picture illustrates the current scenario.

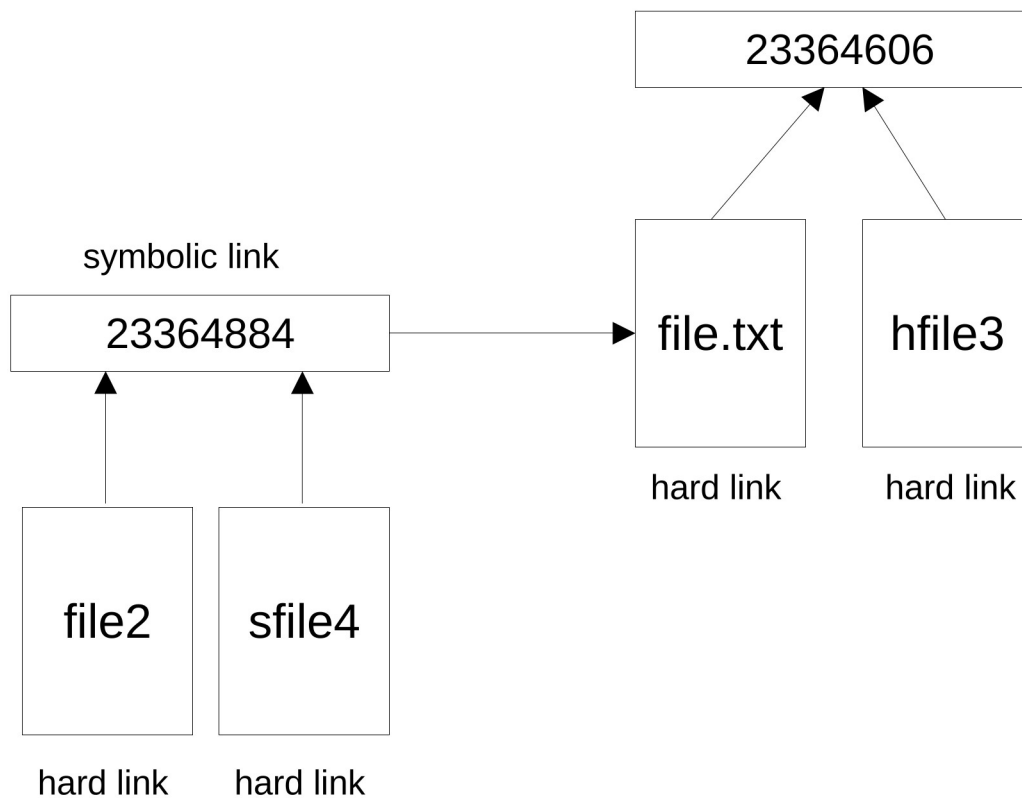


Fig 2. Relationship between symbolic and hard links

`sfile4` (like `file2`) is an hard link to a symbolic link, or better to an inode that is a symbolic link. Both the inodes 23364606 and 23364884 represent a file, but the second represents a file that is a pointer to another file. A symbolic link has no content, is only a pointer to the data of another file.

We view the content af all created files.

```
[linux@fidir] cat *
hello
world
hello
world
hello
world
hello
world
```

We have four references to the same data with only two inodes.

The field `st_nlink` of the `stat` structure represents the number of the hard links of the file.

The field `st_mode` defines type and mode of the file.

On a Linux system, files can be of different types:

Regular file

A file that contains data, it can be text or binary

Directory

An access point in the file system containing other files and directories

Block file

A file that provides buffered I/O access to devices with a fixed-size units of data

Character file

A file that provides unbuffered I/O access to devices with variable-size units of data

FIFO

Used for communication between processes, also called *named pipe*

Socket

Used for network communication between processes and for non-network communication between processes on the same host (*interprocess communication*)

Symbolic link

A file that points to another file

These types are specified in the `st_mode` field as listed in the following table.

<code>S_IFMT</code>	0170000	bit mask for the file type bit field
<code>S_IFBLK</code>	0060000	block device
<code>S_IFCHR</code>	0020000	character device
<code>S_IFDIR</code>	0040000	directory
<code>S_IFIFO</code>	0010000	FIFO
<code>S_IFLNK</code>	0120000	symbolic link
<code>S_IFREG</code>	0100000	regular file
<code>S_IFSOCK</code>	0140000	socket

Tab 4. Values of `st_mode`

The program `getinfo.c` retrieves information about the file supplied on the command line⁷.

```
1 #include <stdint.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/stat.h>
5 #include <sys/sysmacros.h>
6 #include <time.h>

7 int main (int argc, char *argv[])
```

⁷ The code is taken from the `stat(2)` manual page

```

8 {
9     struct stat sb;
10    if (argc != 2) {
11        fprintf(stderr, "Usage: %s <pathname>\n", argv[0]);
12        exit (EXIT_FAILURE);
13    }
14    if (lstat(argv[1], &sb) == -1) {
15        perror("lstat");
16        exit (EXIT_FAILURE);
17    }
18    printf("ID of device: [%x, %x]\n", major(sb.st_dev), \
        minor(sb.st_dev));
19    printf("File type: ");
20    switch(sb.st_mode & S_IFMT) {
21        case S_IFBLK: printf ("block device \n"); break;
22        case S_IFCHR: printf ("character device \n"); break;
23        case S_IFDIR: printf ("directory \n"); break;
24        case S_IFIFO: printf ("FIFO/pipe \n"); break;
25        case S_IFLNK: printf ("symlink \n"); break;
26        case S_IFREG: printf ("regular file \n"); break;
27        case S_IFSOCK: printf ("socket \n"); break;
28        default: printf("unknown? \n"); break;
29    }
30    printf ("I-node number: %ju\n", (uintmax_t) sb.st_ino);
31    printf ("Mode: %jo (octal)\n", (uintmax_t) sb.st_mode);
32    printf ("link count: %ju\n", (uintmax_t) sb.st_nlink);
33    printf ("Ownership: UID=%ju GID=%ju\n", (uintmax_t) sb.st_uid, \
        (uintmax_t) sb.st_gid);
34    printf ("I/O block size: i %jd bytes\n", (intmax_t) sb.st_blksize);
35    printf ("File size: %jd bytes\n", (intmax_t) sb.st_blocks);
36    printf ("Block allocated: %jd\n", (intmax_t) sb.st_blocks);
37    printf ("Last status change: %s", ctime(&sb.st_ctime));
38    printf ("Last file access: %s", ctime(&sb.st_atime));
39    printf ("Last file modifications: %s", ctime(&sb.st_mtime));
40    exit (EXIT_SUCCESS);
41 }

```

getinfo.c

At line 9, the program declares a variable of type `struct stat` named `sb`, which will store the file information returned by `lstat`, and at line 10, it checks whether exactly one pathname argument was provided on the command line.

At lines 11–12, if the pathname is missing, we print a usage message to `stderr` and terminates using `exit (EXIT_FAILURE)`.

At line 14, the program calls `lstat`, passing the pathname provided by the user and the address of the `sb` structure, where the file metadata will be stored. Next, at lines 15–16, if `lstat` fails, we print the corresponding error message using `perror` and terminates with `EXIT_FAILURE`.

At line 18, the program prints the major and minor numbers of the device containing the file by using the functions `major` and `minor`⁸ defined in `sys/sysmacros.h`.

At line 19, it starts to print the type of the file. The program performs a bitwise AND operation between `sb.st_mode` and the constant `S_IFMT`⁹ at line 20, in order to extract the file type information from the mode field.

At lines 21–28, we use a `switch` statement to determine the file type and print the corresponding description, such as regular file, directory, symbolic link, socket, or device. Then, at line 30, we print the inode number stored in `sb.st_ino` after casting it to `uintmax_t`¹⁰.

At line 31, the file mode in octal format is printed after casting `sb.st_mode` to `uintmax_t`.

At line 32, the number of hard links associated with the file is printed using the field `sb.st_nlink`.

After printing the user ID (UID) and group ID (GID) that own the file at line 33, the program prints the preferred I/O block size for the filesystem and at line 35 prints the size of the file.

At line 36, it prints the number of allocated blocks used by the file.

At lines 37–39, it calls `ctime`¹¹ to convert the timestamps stored in `sb.st_ctime`, `sb.st_atime`, and `sb.st_mtime` from `time_t` format into human-readable ASCII strings, and then prints them.

We execute the program passing its source file.

```
[linux@fidir] ./getinfo getinfo.c
ID of device:          [103, 2]
File type:             regular file
I-node number:        23629952
Mode:                 100644 (octal)
link count:           1
Ownership:             UID=1000  GID=1000
I/O block size: i     4096 bytes
File size:             8 bytes
Block allocated:      8
Last status change:   Sun Mar 24 22:06:09 2024
Last file access:     Sun Mar 24 22:06:29 2024
Last file modifications: Sun Mar 24 22:06:09 2024
```

We execute the program passing a device file.

```
[linux@fidir] ./getInfo /dev/tty0
ID of device:          [0, 5]
File type:             character device
I-node number:        13
Mode:                 20620 (octal)
link count:           1
Ownership:             UID=0  GID=5
```

8 On Linux each device has a major and a minor number, the major number represents the device driver (IDE/SATA disk driver, floppy, serial port, etc.), the minor number identifies the device (many devices can share the same device driver)

9 `S_IFMT` is the bitmask for the file type, AND-ing with `sb.st_mode` return the file type. It is defined in `sys/stat.h`

10 `intmax_t` and `uintmax_t` are the signed and unsigned types to represent any value of any integer type supported by the implementation, see the `intmax(3type)` manual page. It is used for the inodes because this can have larger values. These types are defined in `stdint.h`

11 `ctime` takes a pointer as argument, is defined in `time.h`

I/O block size: i	4096 bytes
File size:	0 bytes
Block allocated:	0
Last status change:	Wed Mar 20 17:57:16 2024
Last file access:	Wed Mar 20 17:57:16 2024
Last file modifications:	Wed Mar 20 17:57:16 2024

The program successfully retrieved the `stat` information in both executions.

Permissions

On Linux system (and Unix systems), each file has a set of permissions that define the access mode to the file.

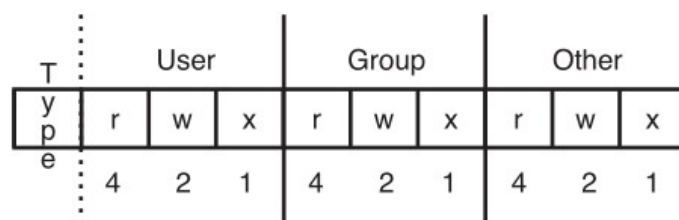


Fig 3. File permissions

File permissions are divided into three categories: the owner (*user*) of the file, the *group* associated with the file, and *others* (all remaining users). The first field indicates the type of the file.

–	regular file
l	symbolic link
b	block device file
c	character device file
d	directory
p	FIFO (named pipe)
s	socket

Tab 5. File types

The second field represents the permissions: the first set (3 elements) defines the *owner's permissions*, the next the *group's permissions*, the last the *others permissions*. Each permission has a numeric and a string value.

r	read	4
w	write	2
x	execute	1

Tab 6. Types of permissions

Read permission means the user can read the contents of the file, *write permission* means the user can modify the file's contents, *execute permission* means the file can be executed. Concerning the directories: read means the user can view the directory's contents¹², write is the ability to add and delete files in the directory¹³, execute means the user can traverse the directory (enter in the directory with the `cd` command)¹⁴. To add and delete files in a directory the user needs *execute* and *write permissions* on it.

The sum of these type specifies the permissions for the user/owner/other, so for example the owner can have a permission of 6 ($6 = 4 + 2$) that means read and write and no execute, the group can have a permission of 5 ($5 = 4 + 1$) that means read and execute but no write. All the possible combinations of 4, 2 and 1 define a specific permission. The lowest permission is 000 that is defined by ---. Where there is no permission, Linux use a - to represent it. The highest permission is 777, where 7 is the sum of $4 + 2 + 1$ which means read, write and execute.

The following table lists the possible combinations of values.

Total	Sum	rwX	Permission
7	$4 + 2 + 1$	rwX	read, write, execute
6	$4 + 2$	rw-	read and write
5	$4 + 1$	r-X	read and execute
4	4	r--	read only
3	$2 + 1$	-wX	write and execute
2	2	-w-	write only
1	1	--X	execute only
0	0	---	none

Tab 7. Combinations of the permissions values

The following table lists some examples.

777	user: read, write, execute; group: read, write, execute; other: read, write execute
664	user: red, write; group: read, write, other: read

¹² To see the files attributes (as shown by `ls -l`) the execution permission is required for the directory

¹³ An user can delete a file in a directory if he has the write permission on that directory even if he is not the owner of that file

¹⁴ To execute `cd /var/log` the user must have the execution permission for all the directories of the path: `/`, `var` and `log`

755	user: read, write, execute; group: read, execute; oher: read, execute
5--	user: read, execute; group and others no permissions
-62	user: no permission; group: read and write; other: write

Tab 8. Example of permissions

To view the file permissions we can use `ls` with the option `-l`

```
[linux@fidir] ls -l
total 8
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 file2 -> file.txt
-rw-r--r-- 2 fidir fidir 12 Mar 23 21:09 file.txt
-rw-r--r-- 2 fidir fidir 12 Mar 23 21:09 hfile3
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 sfile4 -> file.txt
```

`file.txt` and `hfile3` are regular file with 644, `file2` and `sfile4` are symbolic links with 777.

In the previous example we executed our program passing the file `/dev/tty`, the output for the `st_mode` field was

Mode: 20620 (octal)

The leading digit represents the file type/flags, followed by the standard permission values for owner, group, and others. In this case, the owner has permission value 6, the group has permission value 2, and others have permission value 0.

On Linux systems, a regular file is represented in the first bit of the permissions with a `-`.

The *permissions mask* is a bitmask of 10 bit where the first bit represents the type of the file and the others 9 bits represent the three-bit fields of the owner, group and others¹⁵. The field `st_mode` of the structure `stat` is a bit mask containing the permissions and the type of the file. The file's type can be extracted by ANDing the value of the field with the constant `S_IFMT`. In the `getInfo.c` program we compared this values with the constants to print the type of file. To perform this operation are also available the following macros defined in `sys/stat.h`.

Macro	Value	Description
<code>S_ISLNK(m)</code>	<code>((m) & S_IFMT) == S_IFLNK)</code>	Returns true if the file is a symbolic link
<code>S_ISREG(m)</code>	<code>((m) & S_IFMT) == S_IFREG)</code>	Returns true if the file is a regular file
<code>S_ISDIR(m)</code>	<code>((m) & S_IFMT) == S_IFDIR)</code>	Returns true if the file is a directory
<code>S_ISCHR(m)</code>	<code>((m) & S_IFMT) == S_IFCHR)</code>	Returns true if the file is a character device
<code>S_ISBLK(m)</code>	<code>((m) & S_IFMT) == S_IFBLK)</code>	Returns true if the file is a block device
<code>S_ISFIFO(m)</code>	<code>((m) & S_IFMT) == S_IFIFO)</code>	Returns true if the file is a FIFO (named pipe)
<code>S_ISSOCK(m)</code>	<code>((m) & S_IFMT) == S_IFSOCK)</code>	Returns true if the file is a socket

¹⁵ To change the permissions of a file see the `chmod(1)` manual page

Tab 9. Macro to retrieve the type of file

The program `getfiletype.c` retrieves the type of the file supplied on the command line.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/stat.h>
4 int main (int argc, char *argv[])
5 {
6     struct stat sb;
7     if (argc != 2) {
8         printf("\nUsage: %s <file>", argv[0]);
9         exit(EXIT_FAILURE);
10    }
11    if (lstat(argv[1], &sb) == -1) {
12        perror("lstat");
13        exit (EXIT_FAILURE);
14    }
15    int type = sb.st_mode & S_IFMT;
16    if (S_ISREG(type))
17        printf("\n%s: Regular file", argv[1]);
18    else if(S_ISLNK(type))
19        printf("\n%s: Symbolic link", argv[1]);
20    else if(S_ISDIR(type))
21        printf("\n%s: Directory", argv[1]);
22    else if(S_ISCHR(type))
23        printf("\n%s: Character device", argv[1]);
24    else if(S_ISBLK(type))
25        printf("\n%s: Block device", argv[1]);
26    else if(S_ISFIFO(type))
27        printf("\n%s: FIFO", argv[1]);
28    else if(S_ISSOCK(type))
29        printf("\n%s: Socket", argv[1]);
30    return 0;
31 }
```

`getfiletype.c`

At line 6, the program declares a variable of type `struct stat` named `sb`, which will store the file information returned by `lstat`. At line 11, the program calls `lstat` to retrieve information about the file specified on the command line and stores the result in the `sb` structure. At line 15, the program performs a bitwise AND operation between `sb.st_mode` and the constant `S_IFMT` to extract the file type information, and stores the result in the variable `type`. At lines 16–29, the program checks the file type using macros such as `S_ISREG`, `S_ISLNK`, `S_ISDIR`, and others, then prints the corresponding file type description.

We execute the program.

<pre>[linux@fidir] ./getfiletype getfiletype.c getfiletype.c: Regular file</pre>
--

```
[linux@fidor] ./getfiletype /home  
/home: Directory
```

```
[linux@fidor] ./getfiletype /usr/bin/X  
/usr/bin/X: Symbolic link
```

```
[linux@fidor] ./getfiletype /tmp/dbus-5HHov05X  
/tmp/dbus-5HHov05X: Socket
```

By using the `stat` structure, we can develop programs for many different purposes. For example, we can write a program that searches for files with specific permissions, files with a given last modification date, or files whose size is greater or smaller than a specified threshold, and many other similar criteria.